

DATA CLEANING PROCESS

Raw Data is **Lying** to You.

A complete data cleaning walkthrough — 5,050 messy records transformed using SQL & Pandas, step by step.

PostgreSQL

Python · Pandas

5,050 Records

7 Columns

8

Cleaning Steps

50

Exact Duplicates Found

2x

Tools Compared

STEP 00

What the raw data looked like

Before any cleaning — a snapshot of the real problems hiding in the dataset. Every highlighted cell is a data quality issue.

◆ BEFORE CLEANING

id	name	age	email	city
1	NaN	131	invalid_email	Bangalore
3	Rahul	145	rahul@yahoo.com	Chennai
4	Riya	19	riya@outlook.com	MUMBAI
5	Neha	-2	neha@yahoo.com	MUMBAI
47	Rahul	148	rahulgmail.com	delhi

◆ AFTER CLEANING

id	name	age	email
1	Unknown	42	NULL
3	Rahul	42	rahul@yahoo.com
4	Riya	19	riya@outlook.com
5	Neha	42	neha@yahoo.com
47	Rahul	42	rahul@gmail.com

ISSUES FOUND

NaN names · impossible ages (-2, 131, 145, 148) · invalid emails · inconsistent city casing (MUMBAI, delhi, Delhi) · missing salary values

STEP 01

Data Inspection

First rule: never clean blind. Inspect structure, shape, and stats before touching anything.

SQL – PostgreSQL

```
-- row count & preview
SELECT COUNT(*) FROM dataa;
SELECT * FROM dataa LIMIT 10;

-- column types
SELECT column_name, data_type
FROM information_schema.columns
WHERE table_name = 'dataa';
```

Python – Pandas

```
# shape, types, stats
df.head()
df.shape
df.info()
df.describe()

# Result: 5050 rows x 7 cols
# signup_date stored as text!
```

WHY THIS MATTERS

df.head() catches visual errors like "invalid_email". **df.info()** reveals type mismatches — `signup_date` was a string, not a date. **df.describe()** flags outliers instantly (min age = -2, max = 148).

STEP 02

Finding Missing Values

Quantify nulls before deciding strategy. Dropping everything would destroy too much data.

SQL – PostgreSQL

```
SELECT
  COUNT(*) - COUNT(name)
    AS missing_name,
  COUNT(*) - COUNT(age)
    AS missing_age,
  COUNT(*) - COUNT(salary)
    AS missing_salary
FROM dataa;
```

Python – Pandas

```
# count nulls per column
df.isnull().sum()

# Results:
# name      → 1,010 nulls
# salary    → 1,642 nulls
# signup_date → 540 nulls
```

STRATEGY

Dropping nulls removes too much data. Instead: impute **age & salary** with median (robust against outliers), fill **name/city** with "Unknown", and keep **signup_date** null — time fields shouldn't be guessed.

STEP 03

Outlier Handling · Age

Ages of -2 and 148 are biologically impossible. Flag outliers first, then impute with median — not mean.

SQL — PostgreSQL

```
-- flag outliers
SELECT * FROM dataa
WHERE age::int < 0
      OR age::int > 100;

-- null them out
UPDATE dataa
SET age = NULL
WHERE age::int < 0
      OR age::int > 100;

-- fill with avg
UPDATE dataa
SET age = ROUND(
  (SELECT AVG(age::int)
   FROM dataa
   WHERE age IS NOT NULL))
WHERE age IS NULL;
```

Python — Pandas

```
# flag invalid ages → NaN
df_clean.loc[
  (df_clean['age'] < 0) |
  (df_clean['age'] > 100),
  'age'
] = np.nan

# impute with median := 42
df_clean['age'] = (
  df_clean['age'].fillna(
    df_clean['age'].median()
  )
)
```

MEDIAN VS MEAN

Median (42) is preferred over mean when outliers exist. Extreme values like 148 inflate the mean, making imputation biased. Two-step approach: nullify first → impute after.

STEP 04

Duplicate Detection & Removal

The dataset had 5,050 rows but only 50 unique customers — the last 5,000 rows were exact copies. Found using ROW_NUMBER + customer_id.

SQL – PostgreSQL

```
-- find dupes via window fn
WITH cte AS (
  SELECT ctid,
         ROW_NUMBER() OVER(
           PARTITION BY customer_id
           ORDER BY signup_date DESC
         ) AS rn
  FROM dataa
)
DELETE FROM dataa
WHERE ctid IN (
  SELECT ctid FROM cte
  WHERE rn > 1
);
```

Python – Pandas

```
# check dupes
df_clean.duplicated().sum()
# → 5000 duplicates!

# verify by customer_id
df[df_clean.duplicated(
  subset='customer_id',
  keep=False
)].sort_values('customer_id')

# drop exact duplicates
df_clean.drop_duplicates(
  inplace=True
)
```

SQL TRICK

No primary key → used PostgreSQL's internal **ctid** (physical row address) to uniquely identify rows for deletion. Safe and precise without needing a unique column.

STEP 05

Text Standardization

"delhi", "DELHI", "Delhi" — same city, three different strings. Grouping breaks. Fix: trim + title case everywhere.

SQL — PostgreSQL

```
-- remove spaces
UPDATE dataa
SET name = TRIM(name),
    city = TRIM(city),
    email = TRIM(email);

-- title case
UPDATE dataa
SET name = INITCAP(name);

UPDATE dataa
SET city = INITCAP(city);
```

Python — Pandas

```
# strip + title case
df_clean['name'] = (
    df_clean['name']
    .str.strip()
    .str.title()
)
df_clean['city'] = (
    df_clean['city']
    .str.strip()
    .str.title()
)
# delhi → Delhi
# MUMBAI → Mumbai
```

RESULT

City distribution after cleaning: Delhi (1,256) · Mumbai (1,236) · Bangalore (658) · Unknown (645) · Chennai (637) · Kolkata (618). All consistent now.

STEP 06

Email Validation & Repair

Two problems: "rahulgmail.com" (missing @) and "invalid_email" (unfixable). Fix what can be fixed — null the rest.

SQL – PostgreSQL

```
-- fix missing @ sign
UPDATE dataa
SET email = REPLACE(
    email, 'gmail.com',
    '@gmail.com')
WHERE email LIKE '%gmail.com'
AND email NOT LIKE '%@%';

-- null unfixable emails
UPDATE dataa
SET email = NULL
WHERE email NOT LIKE '%@%.%';
```

Python – Pandas

```
# smart repair function
def fix_email(email):
    if isinstance(email, str) \
        and '@' not in email:
        for domain in [
            'gmail.com', 'yahoo.com',
            'outlook.com']:
            if domain in email:
                return email.replace(
                    domain, '@'+domain)
        return email

df_clean['email'] = \
    df['email'].apply(fix_email)
```

DECISION LOGIC

Recoverable → fix (add missing @). Unrecoverable → NULL, not a fake value. Never invent data. Keeping "invalid_email" as-is would silently corrupt downstream analysis.

STEP 07

Data Type Correction

All columns loaded as text. Wrong types break aggregations, date filters, and sorting. Cast everything to its true type.

SQL – PostgreSQL

```
ALTER TABLE dataa
ALTER COLUMN customer_id
  TYPE INTEGER
  USING customer_id::INTEGER;

ALTER TABLE dataa
ALTER COLUMN age
  TYPE INTEGER
  USING age::INTEGER;

ALTER TABLE dataa
ALTER COLUMN signup_date
  TYPE DATE
  USING signup_date::DATE;
```

Python – Pandas

```
# nullable int (handles NaN)
df_clean['age'] = (
    df_clean['age'].astype('Int64'))

df_clean['salary'] = (
    df_clean['salary'].astype('Int64'))

# parse date, coerce errors
df_clean['signup_date'] = (
    pd.to_datetime(
        df_clean['signup_date'],
        errors='coerce'))
```

PANDAS TIP: INT64 VS Int64

Standard **int64** crashes when NaN exists. Use nullable **Int64** (capital I) — it holds integers AND null values. **errors='coerce'** on date parsing converts bad values to NaT silently instead of throwing errors.

FINAL

Cleaning Pipeline Complete



Inspect First

`.info()` + `.describe()` + `information_schema` — always understand your data before touching it.



Nullify, Then Impute

Flag outliers → convert to null → fill with median. Two-step prevents biased imputation.



Deduplicate Carefully

`ROW_NUMBER` + `ctid` (SQL) or `drop_duplicates()` (Pandas). Verify before deleting — 5,000 dupes removed.



Fix What's Fixable

Recover emails with missing `@`. Null unrecoverable values. Never invent data to fill gaps.



Standardize Text

`TRIM` + `INITCAP` (SQL) / `.str.strip().str.title()` (Pandas). City casing silently broke groupby.



Cast Types Last

Clean data first, then cast. Wrong order causes type conversion errors on dirty values.

SQL + Pandas — same problem, two tools

Which approach do you prefer for data cleaning?
Drop your answer in the comments 🙋